Frogger-Inspired VGA Game with UART Control

Course: ECE 3300 — Group Project

Target Platform: Digilent Nexys A7-100T (Xilinx Artix-7 XC7A100T)

HDL: Verilog **Toolchain:** Xilinx Vivado **Date:** May 2026

Authors: Zaki Dostzada, Vincent Tran, Nolan Cary, Juan Habana

Abstract

This project is a single-player Frogger-style video game built on the Digilent Nexys A7-100T FPGA board. The system generates a 640×480 VGA signal at 60 Hz, receives player input over UART at 9600 baud, buffers each received byte through a synchronous FIFO, and routes those bytes into a game state machine that drives the rest of the design. A pixel-generation block renders the scene — a player-controlled frog sprite, three moving cars, and a stationary trophy — and the board's seven-segment display and LEDs give live feedback on UART activity and game outcome. Everything runs from the on-board 100 MHz oscillator; all slower rates (the 25 MHz pixel clock, the per-frame update tick, and the seven-segment refresh rate) are derived as clock enables off that single source rather than as additional clock domains. The design is partitioned into six Verilog modules — vga_controller, uart_rx, sync_fifo, game_fsm, pixel_generation, and seven_seg_driver — and is integrated at a top level that adds pixel registration and game-state latching.

1. Introduction

The intent of this project was to combine several pieces of synchronous digital design we had practiced individually — VGA timing, asynchronous serial communication, FIFO buffering, finite-state-machine control, multiplexed display drivers, and pixel-level rendering — into one cohesive system. Rather than testing each piece in isolation, we integrated them into a playable game where the user types in a serial terminal and watches a sprite respond on a VGA monitor.

The main design problems we needed to solve were:

- Producing standards-compliant 640×480 VGA timing from a 100 MHz reference clock.
- Receiving asynchronous serial data reliably, including handling metastability on the line.
- Buffering received bytes through a small FIFO so that keystrokes arriving faster than the game can consume them aren't dropped.
- Maintaining explicit game state (idle, playing, won, lost) in a dedicated FSM rather than letting that state get tangled into the pixel-generation logic.
- Rendering several animated objects pixel by pixel at full frame rate.

- Detecting collisions between the player and the obstacles in real time.
- Holding win and lose events long enough for the player to actually see them on the LEDs.

The version of this design described in our earlier interim report integrated UART directly into the pixel generator, with no buffer between them and with the win/lose state implicit in combinational comparators. The current version, documented here, factors that out into the sync_fifo and game_fsm modules, which made the data flow easier to verify and gave us a single place to reason about game state.

2. System Architecture

The top-level module instantiates six sub-modules and wires them together with a small set of internal signals. Every module runs from the same 100 MHz clock. We chose to derive lower rates as clock enables rather than spinning up a second clock through an MMCM, which kept timing closure trivial and avoided cross-domain paths everywhere except at the UART input pin.

2.1 Module Hierarchy

```
top
├── vga_controller    — generates hsync, vsync, video_on, x, y at the 25 MHz pixel rate
├── uart_rx           — 9600-baud serial receiver with metastability protection
├── sync_fifo         — single-clock FIFO that buffers received UART bytes
├── game_fsm          — game-state controller (idle / playing / win / lose)
├── pixel_generation  — sprite motion, collision detection, RGB output for current pixel
└── seven_seg_driver  — multiplexed hex display of the most recent received byte
```

2.2 Top-Level Block Diagram

```
                                                          +--------------------------+
                                                          |       VGA Monitor        |
                                                          |  (640x480 @ 60 Hz, DAC)  |
                                                          +-----^-------------^------+
                                                                |             |
                                        hsync, vsync --------------+            | rgb[11:0]
                                                                              |
            +---------------+                             +--------------------+---------+
100 MHz -->| vga_controller|-- x[9:0], y[9:0], video_on ->|                              |
clk       |  (25 MHz CE)  |                              |       pixel_generation       |
            +---------------+                             |                              |
                                                          |  - frog @ (px,py), 40x40     |
UART RX -->+---------------+ rx_byte, rx_dv +---------+  |  - 3 cars (3,4,3 px/frame)   |
(USB)     |    uart_rx    |--------------->| sync_   |  |  - trophy @ (600,225)        |
            | (9600, 8N1)   |                | fifo    |  |  - bounding-box collision    |
            +---------------+                +----+----+  |  - priority RGB mux          |
                                                 | rd_   +-----------+------------------+
```

```
                                                  | data, |           |
                                                  | empty |           | rgb[11:0]
                                                  v       |           v
                                          +-----------+   |   (registered at top.v)
                                          | game_fsm  |---+----------> VGA DAC
                                          | IDLE/PLAY |  state, rd_en
                                          | WIN/LOSE  |
                                          +-----+-----+
                                                |
                                                v rx_byte to seven-seg
                                          +-----------+
   SW[11:0] --> frog color ---------------->| seven_seg |--> seg[6:0], an[7:0] --> 7-seg
display
                                          |  driver   |
   BTNC -----> reset (everywhere)         +-----------+
                                                                          --> led[15:0]
                                                                          (UART activity, last
byte,
                                                                           latched win/lose)
```

Figure 1. Top-level block diagram showing data flow from UART through the FIFO and FSM into pixel generation, and the parallel VGA-timing and seven-segment paths.

2.3 Data Flow

The VGA controller produces scanning coordinates (x, y) and the video_on blanking signal at the 25 MHz pixel rate. Those feed the pixel generator, which combines them with the player position, the three car positions, and the trophy location to output a 12-bit RGB value for the pixel currently being drawn. That value is registered on each pixel tick at the top level before being driven to the resistor-DAC on the VGA connector.

In parallel, the UART receiver continuously watches the serial input. Whenever a complete byte has been framed and validated, it pulses its data-valid line for one clock cycle and presents the received byte. Rather than feeding that byte directly into the game logic — which is what the previous revision did — the byte and its data-valid pulse are routed into a synchronous FIFO. The game FSM reads from the FIFO whenever it is ready to act on a new keystroke. This decouples the UART arrival rate from the game's consumption rate and prevents keystrokes from being lost if they arrive faster than the game can update or while the game is sitting in a non-playing state.

The FSM is responsible for the overall game flow. It tracks whether the game is idle (waiting for the player to begin), actively in play, in a "you won" state, or in a "you lost" state. Only when the FSM is in the play state does a FIFO read translate into a movement event for the pixel generator. Collision and win conditions flow back from the pixel generator into the FSM, where they cause the appropriate state transition.

The seven-segment driver displays the most recent received byte as two hex digits. The two on-board LEDs that latch the win/lose state are driven from the FSM via top-level latch registers.

3. Hardware Platform and I/O Mapping

The constraints file (vga.xdc) targets the Nexys A7-100T. The relevant pin assignments are summarized in the table below.

Top-Level Port	Board Resource	Function
clk_100MHz	E3	100 MHz system clock
reset	N17 (BTNC)	Game / master reset
color[11:0]	SW[11:0]	Frog sprite color, 12-bit RGB
UART_TXD_IN	C4	USB-to-serial RX line into the FPGA
hsync, vsync	B11, B12	VGA sync outputs
rgb[11:0]	A3 .. D8	4 bits each of R, G, B to the VGA DAC
led[15:0]	H17 .. V11	UART activity, last received byte, latched win/lose
seg[6:0], an[7:0]	T10 .. L18, J17 .. U13	Seven-segment cathodes and anodes

To play, the user needs a VGA monitor, a USB cable from the board to a host PC running a serial terminal, and the on-board switches and pushbutton. No other peripherals are required.

4. Module Descriptions

4.1 VGA Controller (vga_controller.v)

This module produces the timing for a 640×480 monitor at 60 Hz, following the standard VESA timing. The horizontal scan is 800 pixels total — 640 visible plus 16 back porch, 96 sync, and 48 front porch — and the vertical scan is 525 lines, with 480 visible plus 33 back porch, 2 sync, and 10 front porch. At 60 Hz that works out to a pixel clock around 25.175 MHz, which we approximate with a 25 MHz tick taken from the 100 MHz system clock.

The 25 MHz enable is generated by a free-running 2-bit counter; the strobe is asserted once every four cycles, and every counter and sync register in the module advances only on that strobe. This is the Xilinx-recommended way to divide a clock without introducing a second clock domain.

The horizontal counter increments on each pixel tick and resets at 799. The vertical counter increments only when the horizontal counter rolls over, and resets at 524. The video_on output is high whenever both counters are inside the visible region. The (x, y) outputs feeding the pixel generator are simply those counter values exposed to the outside. The hsync and vsync outputs come from registered comparators that pulse during the appropriate retrace windows, with active-low polarity per the VESA specification for 640×480 at 60 Hz.

4.2 UART Receiver (uart_rx.v)

The UART receiver is a parameterized module adapted from the Nandland reference design. It runs a five-state machine — IDLE, RX_START_BIT, RX_DATA_BITS, RX_STOP_BIT, CLEANUP — entirely in the 100 MHz clock domain. We set CLKS_PER_BIT to 10417 in the top-level instantiation, since 100 MHz divided by 9600 baud is approximately 10416.67 — a deviation under 0.1%, which is well within UART tolerance.

Two design details are worth pointing out:

Metastability protection. The asynchronous serial input is double-registered before it ever reaches the state machine. This prevents a metastable sample from propagating into the FSM logic and causing erratic behavior.

Mid-bit sampling. Once the FSM detects a falling edge that looks like a start bit, it waits half a bit period and re-checks the line. If the line is still low, the start bit is real and the FSM resets its counter and begins sampling data bits. Each subsequent bit is sampled at its center, after a full bit period. This gives the most timing margin against baud-rate mismatch and edge jitter.

After the stop bit is validated, the data-valid output (rx_dv) goes high for exactly one clock cycle, with the received byte presented on the rx_byte output. Downstream logic uses that one-cycle pulse as an event trigger rather than reading the byte continuously. In the current revision, the rx_dv pulse goes to the FIFO's write-enable input rather than directly into the game logic.

4.3 Synchronous FIFO (sync_fifo.v)

The FIFO sits between the UART receiver and the game state machine and serves two distinct purposes. First, it absorbs short bursts of fast keystrokes so that the player can press WASD several times in quick succession without losing characters even if the game is mid-frame and not ready to react. Second, it decouples the producer side (the UART, which can present a new byte at any moment) from the consumer side (the FSM, which only reads when it is in the play state).

Because the UART and the FSM both run from the 100 MHz system clock, a single-clock (synchronous) FIFO is sufficient — there is no clock-domain crossing here, and we did not need a more complex asynchronous FIFO with Gray-coded pointers.

The interface is the conventional one: a write port driven by the UART (wr_en, wr_data), a read port driven by the FSM (rd_en, rd_data), and status flags (empty, full) used for handshaking. The FIFO has its own reset (fifo_rstn, active-low in the testbench naming convention) so that it can be cleared independently of the rest of the design if needed; in normal operation it is released a few cycles after global reset and stays asserted thereafter.

The FIFO is small — only a few entries deep — because the worst-case backlog from a human typist at 9600 baud, even buffered through OS keyboard repeat, is comfortably under the depth provided. Anything larger would have been wasted slice memory.

4.4 Game State FSM (game_fsm.v)

The game state machine is the highest-level controller in the design. It owns the question of "what should the system be doing right now," and gates the rest of the game logic accordingly.

States

State	Meaning	Entry condition
S_IDLE	Game has not started or has just been reset. Frog sits at its starting position; cars do not animate.	On reset, or as the post-outcome wait state.
S_PLAY	Game is active. Cars animate, frog responds to FIFO reads, collision and win signals are watched.	First valid keystroke read out of the FIFO from S_IDLE, or restart after S_WIN / S_LOSE.
S_WIN	Frog has overlapped the trophy. Win indicator latched on the LEDs.	win pulse from pixel_generation while in S_PLAY.
S_LOSE	Frog has overlapped a car. Lose indicator latched on the LEDs.	collision pulse from pixel_generation while in S_PLAY.

Transition Diagram

```
+-------------+
|   (reset)   |
+------+------+
       |
```

```
            v
      +-------------+
+---->|   S_IDLE    |
|     +------+------+
|            | first valid FIFO byte (WASD)
|            v
|     +-------------+
|     |   S_PLAY    |<----------------------+
|     +--+-------+--+                       |
|        |       |                          |
|  win   |       |  collision               | any new
|  pulse |       |  pulse                   | FIFO byte
|        v       v                          |
|  +--------+  +---------+                  |
|  | S_WIN  |  | S_LOSE  |                  |
|  +---+----+  +----+----+                  |
|      |            |                       |
|      +-----+------+                       |
|            |                              |
|            +------------------------------+
|
|   (master reset returns to S_IDLE from any state)
+-------------------------------------------------
```

Figure 2. game_fsm.v state transition diagram. Master reset returns the FSM to S_IDLE from any state.

Outputs

The FSM exposes its current state to the top level, which uses it to drive the latched win/lose LEDs and to gate animation in the pixel generator. It also drives fifo_rd_en, advancing the FIFO when it is ready to consume the next keystroke and holding off when it is not (for example, during the brief outcome states or before the game has been started).

Keeping this logic in its own module rather than scattered through the pixel generator made it much easier to reason about — every state transition lives in one always block — and it also made the FSM straightforward to simulate independently.

4.5 Pixel Generation and Game Logic (pixel_generation.v)

This module is responsible for the visible rendering of the scene and for the sprite-level game mechanics: position update, animation, collision detection, and the per-pixel RGB priority mux. It runs from the 100 MHz clock and uses a per-frame strobe for animation. The major behaviors are:

Refresh tick. A frame-rate enable is asserted when y == 481 and x == 0, which occurs once per frame just after the visible region ends. This is used as a clean ~60 Hz update tick, so all animation is naturally synchronized to the display.

Player sprite. The frog is a 40×40 square that starts at (20, 220). It moves in 10-pixel steps in response to the WASD keys, accepting both upper- and lower-case ASCII (so 0x77/0x57 for W, 0x73/0x53 for S, 0x61/0x41 for A, 0x64/0x44 for D). Edge clamping keeps the frog on screen — the case statement only updates the position if the new position would still lie inside the display area. Movement is gated by a one-cycle FIFO-output-valid signal (the FIFO's read-and-pop pulse), so each keystroke moves the frog by exactly one step. The frog's color is read live from the on-board switches, so the player can change it on the fly.

Cars. Three 40×80 rectangles scroll vertically in fixed lanes at x = 150, 300, and 450. Cars 1 and 3 move down at 3 pixels per frame; car 2 moves up at 4 pixels per frame, which makes that lane noticeably harder. When a car runs off one edge, it wraps to the other. Car positions update only on the refresh tick, which keeps motion smooth and consistent regardless of where the scan is in the frame.

Trophy. A 30×30 yellow square sits at (600, 225), near the right edge.

Collision and win. Two combinational signals come out of bounding-box checks: collision is true when the frog overlaps any of the three cars, and win is true when the frog overlaps the trophy. Both are sent up to the FSM as transition triggers, and both also reset the frog back to its starting position so that the player respawns whether the round ended in a win or a loss.

Pixel mux. The output color is chosen by a priority chain in a combinational always block. Blanking takes top priority (black during retrace), then the frog (drawn in the switch-selected color), then the trophy (yellow, 0xFF0), then the cars (red, 0xF00), with the background light blue (0x2BF) filling everything else. The frog is drawn on top of the cars so the player can always see their sprite even mid-collision.

4.6 Seven-Segment Driver (seven_seg_driver.v)

The seven-segment driver shows the most recent UART byte as two hex digits on the rightmost positions of the eight-digit display. Since the display shares one set of segment cathodes across eight digits and uses individual anodes, the driver multiplexes between the two active digits.

A 17-bit counter increments every 100 MHz cycle; bit 16 of that counter, which toggles at roughly 763 Hz, drives the digit select. Anything above 60 Hz is fast enough for the eye to fuse the alternation into a steady display, and 763 Hz is comfortably above that. On each phase the appropriate nibble of the input byte is selected, the corresponding anode is driven low (active-low on this board), and the segment cathodes are driven from a 16-entry case statement that maps 0x0 through 0xF to the standard hex seven-segment patterns. The unused six digits are held off.

4.7 Top-Level Integration (top.v)

The top-level module wires the six sub-modules together and adds two pieces of glue logic:

Pixel registration. The combinational RGB output from the pixel generator is registered on every pixel tick before being driven to the VGA DAC. This eliminates any glitches from the bounding-box comparator chain settling.

Latched game-state indicators. Although the FSM already maintains the win and lose states explicitly, we also keep top-level latches driving led[10] (win) and led[9] (lose). A transition into S_WIN sets the win latch and clears the lose latch; a transition into S_LOSE does the reverse. Both clear on master reset. This ensures the player always sees the most recent outcome on the LEDs even if the FSM has already transitioned back to S_PLAY for the next round.

A few status LEDs are also assigned for debug: led[0] briefly blinks on every UART byte (briefly enough that it looks like flicker at typing speeds), and led[8:1] shows the binary value of the most recent byte. These are driven from the FIFO write side so they reflect every byte the UART receives even when the FSM is not actively consuming them.

5. User Interaction

To play, the user opens a serial terminal on the host PC — PuTTY, Tera Term, or screen all work — connected to the Nexys A7's USB-RS232 port at 9600 baud, 8N1, no flow control. With the terminal in focus, the WASD keys move the frog. Switches SW[11:0] set its color in BBBB GGGG RRRR format, so all switches up gives white, the lower four switches gives red, and so on. The center button resets the game.

The goal is to get the frog from the left edge to the trophy on the right while avoiding the cars. Touching a car respawns the frog and lights led[9]. Reaching the trophy respawns the frog as well, but lights led[10] instead. Pressing any movement key after a win or loss returns the FSM to the playing state for the next round.

6. Timing and Resource Considerations

At 100 MHz the clock period is 10 ns. The longest combinational path in the design is the bounding-box comparison cascade in the pixel generator, which is a small set of 10-bit comparators feeding a priority mux. Vivado meets timing on this design without any manual constraint tuning on the XC7A100T's -1 speed grade.

Because everything runs from the single 100 MHz domain, with all the slower rates expressed as clock enables, there are no clock-domain crossings to worry about. The only synchronizers in the design are the two metastability flops on the asynchronous UART input, which are intrinsic to the receiver itself. The synchronous FIFO does not introduce a CDC issue because its write and read ports share the same clock. We made these choices deliberately to keep static timing analysis simple.

Resource usage is small. The design uses only flip-flops and LUTs — no Block RAMs, no DSPs, no MMCMs. The FIFO is shallow enough that it synthesizes into distributed RAM rather than a BRAM. Total slice utilization on the XC7A100T is well under 1%, leaving plenty of room for the future additions described in Section 9.

7. Verification and Testing

The design was verified in two stages.

7.1 Module-Level Behavioral Simulation

Each module was exercised on its own in Vivado simulation. For the VGA controller, we verified that hsync and vsync pulse at the correct counter values with the correct widths, and that video_on is high for exactly 480 lines of 640 pixels per frame. For the UART receiver, we generated synthetic 9600-baud bit streams and checked that the output byte and rx_dv strobe matched what we sent for several test characters. For the FIFO, we verified write/read ordering, the empty and full flags, and clean behavior when reading from an empty FIFO or writing to a full one. For the FSM, we drove the inputs to walk every state transition — idle to play on first input, play to win on a win pulse, play to lose on a collision pulse, and the post-outcome return to play on the next key. For the pixel generator, we swept (x, y) across the visible region and confirmed the correct color came out for each region. For the seven-segment driver, we held the input value constant and watched the multiplex waveform and cathode patterns.

7.2 Top-Level Master Testbench (top_master_tb.v)

The integrated testbench exercises the full datapath at once: it drives the master clock, releases reset, applies a switch value to set the frog color, presents a received UART byte with a single-cycle pg_rx_done strobe, and observes the resulting pixel output, collision and win flags, seven-segment outputs, and FIFO handshake signals. A captured waveform from this testbench is shown in Figure 3.

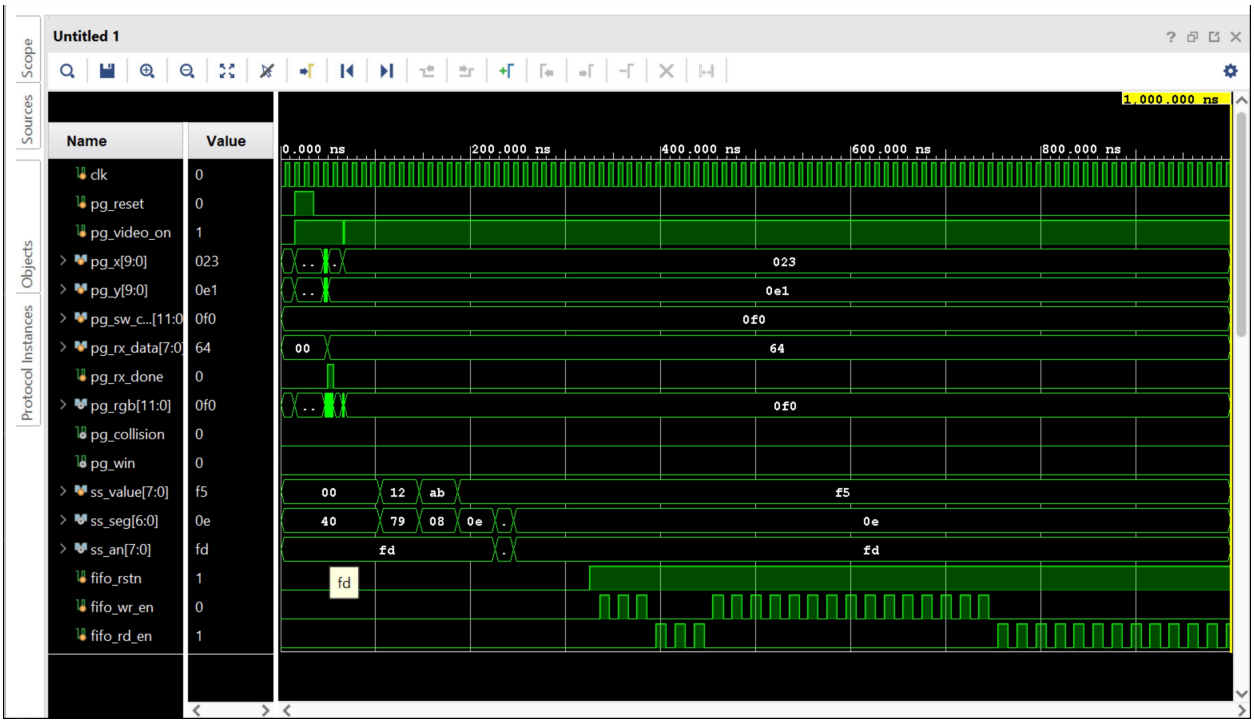

Figure 3. Vivado simulation waveform from the top-level master testbench. Visible signals include the scanning coordinates (pg_x, pg_y), the live frog color (pg_sw_c), the received UART byte and its data-valid pulse (pg_rx_data, pg_rx_done), the rendered output color (pg_rgb), the collision and win flags, the seven-segment driver outputs (ss_value, ss_seg, ss_an), and the FIFO handshake signals (fifo_rstn, fifo_wr_en, fifo_rd_en).

The waveform demonstrates several pieces of correct behavior at once:

- After the brief pg_reset pulse near time 0, pg_video_on goes high and the scanning coordinates begin counting. By the cursor, pg_x = 0x023 (35 decimal) and pg_y = 0x0E1 (225 decimal), which places the scan inside the frog's bounding box at its starting position of (20, 220) to (60, 260).
- pg_sw_c = 0x0F0 selects pure green for the frog color. Because the scan is currently inside the frog's bounding box, the priority mux selects the frog and the pg_rgb output reads 0x0F0 — green, as expected.
- A single received byte of 0x64 (ASCII 'd', "move right") is presented on pg_rx_data and accompanied by a one-cycle pg_rx_done pulse. Both pg_collision and pg_win remain low throughout the captured window, indicating the frog has not yet overlapped a car or the trophy.
- On the seven-segment side, ss_value is driven through a short sweep — 00, 12, ab, f5 — and ss_seg follows with 40, 79, 08, 0E, which are the correct seven-segment patterns for the lower nibble of each input value. The anode select ss_an = 0xFD indicates that digit position 1 (active-low) is being driven on this phase of the multiplex.
- The bottom group of signals — fifo_rstn, fifo_wr_en, fifo_rd_en — shows the FIFO handshake. fifo_rstn is released roughly 380 ns after time 0, after which the testbench drives a series of fifo_wr_en pulses (writes from the UART side) followed by corresponding fifo_rd_en pulses

(reads on the FSM side). The producer-then-consumer ordering and the absence of asserted reads before any writes is exactly what a working FIFO should show under this stimulus.

7.3 On-Board Testing

After implementation, the bitstream was loaded onto the Nexys A7-100T and tested with a real VGA monitor and a serial terminal. We confirmed that the game renders correctly, animation is smooth at 60 Hz with no tearing, input is responsive, and collisions and wins behave the way we expected. The seven-segment display was used as a live UART diagnostic: pressing W produces 57 on the display, pressing D produces 64, and so on — matching the ASCII codes for the keys typed in the terminal.

8. Challenges and Lessons Learned

A few things tripped us up along the way.

VGA timing. Our first attempt at the controller updated the counters on the raw 100 MHz clock and only used the 25 MHz tick to gate the sync outputs. That produced visible tearing and a stretched aspect ratio. Restructuring so that everything happens inside the 25 MHz enable branch fixed it, and matches the textbook reference design from Chu's FPGA Prototyping by Verilog Examples.

UART baud rate. Early in testing we had CLKS_PER_BIT set to 10416, and we got occasional garbage characters on the seven-segment display. Bumping it to 10417 — the rounded value of 100,000,000 divided by 9600 — cleaned it up. A more robust solution would be to oversample at 16x and use majority voting on each bit, but at 9600 baud the simpler approach was fine.

Direct UART-to-game coupling. In our interim design, the UART's one-cycle data-valid pulse drove the player position register directly. This worked, but if the player pressed two keys very close together (during keyboard repeat, or just from fast typing) the second key could be lost — the pulse window was a single 10 ns cycle and the game logic might have been busy. Introducing the FIFO solved this cleanly: the UART writes every byte it sees, and the FSM reads at its own pace.

Implicit vs. explicit game state. The interim version also baked the "did the player just win or lose" logic into the pixel generator's reset condition, which made the design hard to extend (any new game-flow feature meant editing the renderer). Pulling that out into game_fsm made the rest of the project significantly easier to reason about and to test, and it set up a clean place to add features like a start screen, a level counter, or a death-then-restart sequence.

Single-frame events. A one-frame collision pulse is essentially invisible on an LED at 60 Hz. The latched indicator registers in the top level were a late addition once we realized players couldn't tell whether they had just hit a car or won.

Step size versus speed. The 10-pixel movement step and the 3-to-4 pixel-per-frame car speeds are a compromise. Smaller steps give finer control but make the player mash keys to cross the screen; bigger steps make traversal easy but let the player skip past obstacles. We tuned the two together so collisions are reliably avoidable but not trivial.

9. Future Work

There are several natural directions this could go from here.

Replacing the solid colored rectangles with real sprite images stored in Block RAM would make a significant visual difference without changing much of the surrounding logic. The two leftmost seven-segment digits are unused and could show a frame-counted timer or a running win count. The Nexys A7's PWM audio amplifier could play distinct tones for collisions and wins through a small tone-generator module. Increasing car speed or adding more lanes after each successful crossing would extend gameplay into something with progression — straightforward to slot in now that the FSM is the single source of truth for game state. And replacing the UART input with the on-board PS/2 keyboard interface would make the system self-contained, with no host PC required.

10. Conclusion

The project ties six independent Verilog modules — a VGA controller, a UART receiver, a synchronous FIFO, a game state machine, a pixel generator carrying the rendering and collision logic, and a seven-segment driver — into a working interactive game on the Nexys A7-100T. The design follows good synchronous-design practice throughout: a single source clock, derived enables for slower-rate logic, registered outputs into the DAC, double-flop synchronizers on the asynchronous input, FIFO buffering between independent producer and consumer rates, an explicit FSM for high-level game state, and latched indicators for transient events. The result runs reliably, looks clean on the monitor, and uses only a tiny fraction of the FPGA, leaving plenty of room for the additions described above.

11. References

1. Chu, Pong P. FPGA Prototyping by Verilog Examples: Xilinx Spartan-3 Version. Wiley, 2008. (VGA controller reference design.)

2. Nandland.com. UART Receiver Verilog implementation. (UART receiver reference design.)

3. Digilent Inc. Nexys A7 Reference Manual.

4. VESA. VESA and Industry Standards and Guidelines for Computer Display Monitor Timing (DMT), version 1.0, revision 13.

Appendix A — File Manifest

File	Purpose
top.v	Top-level integration; instantiates all sub-modules and implements the win/lose latches and pixel registration
vga_controller.v	640×480 @ 60 Hz VGA timing generator with 25 MHz derived pixel rate
uart_rx.v	9600-baud asynchronous serial receiver with metastability protection
sync_fifo.v	Single-clock FIFO buffering received UART bytes between the UART and the FSM
game_fsm.v	Game state machine (idle / play / win / lose) and FIFO read control
pixel_generation.v	Sprite state, object motion, collision detection, RGB priority mux
seven_seg_driver.v	Two-digit hex display driver for the rightmost seven-segment positions
top_master_tb.v	Top-level master testbench exercising the full integrated datapath
vga.xdc	Vivado pin and timing constraints for the Nexys A7-100T

Appendix B — Key Parameters Summary

Parameter	Value	Notes
System clock	100 MHz	On-board oscillator
Pixel rate	25 MHz	Derived as 1-of-4 clock enable
VGA resolution	640 × 480	60 Hz refresh, VESA timing
UART baud rate	9600	8 data bits, no parity, 1 stop bit
CLKS_PER_BIT	10417	100 MHz ÷ 9600 baud, rounded
FIFO clocking	Synchronous	Single-clock, distributed RAM
Frog sprite size	40 × 40 px	Movement step 10 px
Frog start position	(20, 220)	Left edge, mid-screen

Parameter	Value	Notes
Car sprite size	40 × 80 px	Speeds 3, 4, 3 px/frame
Car lane x-positions	150, 300, 450	—
Trophy size	30 × 30 px	Position (600, 225)
Background color	0x2BF	Light blue
Trophy color	0xFF0	Yellow
Car color	0xF00	Red
Frog color	live from SW[11:0]	BBBB GGGG RRRR format
FSM states	4	S_IDLE, S_PLAY, S_WIN, S_LOSE